

Helping Hands Platform

Technical Documentation

This document explains what the Helping Hands platform is built from and how it works under the hood: architecture, technology choices, database design, security mechanisms, and every major module. It's written for a developer picking up this codebase for the first time, or for anyone evaluating the technical soundness of the build.

Table of Contents

- 1. System Architecture
- 2. Technology Stack
- 3. Backend Architecture
- 4. Frontend Architecture
- 5. Database Design
- 6. Authentication & Security
- 7. Core Modules
- 8. Email System
- 9. Notification System
- 10. API Overview
- 11. Frontend Design System
- 12. Testing
- 13. Deployment & Infrastructure
- 14. Environment Configuration
- 15. Known Limitations & Next Steps

1. System Architecture

Three-tier architecture, deployed as three independently-hosted pieces:

```
React SPA (Vercel) --HTTPS/JSON--> Spring Boot API (Render) --JDBC--> PostgreSQL (Neon)
```

The frontend never talks to the database directly — every read and write goes through the API, which is the only component holding database credentials. Documents (images, verification certificates) are stored on the API server's local disk, encrypted at rest, with metadata in Postgres.

This split across three managed platforms was a deliberate choice for this build (each has a workable free tier), but the backend also ships with a docker-compose.yml for single-VPS self-hosting — the same codebase, no architectural changes needed.

2. Technology Stack

Layer	Technology
Frontend	React 18 + Vite, React Router, Axios, Recharts, lucide-react icons
Backend	Spring Boot 3 (Java 17) — Web, Security, Data JPA, Validation, Mail, Scheduling
Database	PostgreSQL (hosted on Neon)
Auth	JWT (JJWT library), Spring Security, BCrypt password hashing
Email	Pluggable — Resend, SendGrid, or Brevo (HTTPS APIs), or SMTP for self-hosting
File storage	Local disk with AES-256-GCM encryption at rest (interface-based, swappable for cloud storage)
API docs	springdoc-openapi (live Swagger UI)
PDF export	Apache PDFBox
Testing	JUnit 5 + Mockito (backend), Vitest + React Testing Library (frontend)
Hosting	Vercel (frontend), Render (backend), Neon (database)

3. Backend Architecture

Clean Architecture, four layers, dependencies flow one direction only (outer layers depend on inner ones, never the reverse):

```
domain/          Entities only - no Spring dependencies, no persistence logic
                  beyond JPA mapping
application/      Services (business logic) and DTOs (the only objects that
                  cross the API boundary)
infrastructure/   Repositories, security (JWT), storage, email - anything
                  touching the outside world
api/              Controllers (thin - validate, call a service, wrap the
                  response) + exception handling
```

This structure is what allowed roughly 15 distinct modules to be added over the course of the build without ever needing to restructure earlier ones — a controller never contains business logic, and a service never knows what shape the HTTP request took.

4. Frontend Architecture

components/	Reusable UI pieces (StatCard, Pagination, modals, charts, document upload, etc.)
pages/	One file per route, owns its own data-fetching
services/	One file per backend resource - thin axios wrappers; pages never call axios directly
context/	AuthContext (session/roles) and ModalContext (themed confirm/prompt/alert dialogs)
hooks/	useAuth, useModal - thin wrappers around the contexts above
routes/	ProtectedRoute - auth + role gating

Role-based UI (what a Donor sees vs. an Administrator) is driven entirely by reading roles out of the JWT via AuthContext — the same source of truth the backend uses for its own authorization checks, so the two can't drift apart.

5. Database Design

Eleven migrations, applied in order, build the schema:

Migration	Adds
001	users, roles, user_roles
002	Seeds the four original roles
003	childrens_homes, service_providers, service_provider_categories
004	documents (polymorphic owner: home / provider / request)
005	requests, request_status_history
006	ratings, audit_log; suspension columns on users
007	verification_tokens (email verification + password reset)
008	Content-flagging columns on requests
009	notifications
010	DELIVERY_VOLUNTEER role, delivery-method fields, MFA token type, failed_login_attempts, wider notification types
011	resubmission_count on childrens_homes and service_providers

Every business table follows the same platform-wide convention: `is_active`, `created_by`, `created_date`, `modified_by`, `modified_date` — enforced once via a shared BaseEntity that every entity extends. Nothing is ever hard-deleted; suspension, rejection, and moderation are all soft-delete or status-flag operations.

5.1 Key relationships

- User 1:1 ChildrensHome, User 1:1 ServiceProvider, User M:N Role
- ChildrensHome 1:N Request; Request N:1 User (pledgedBy, nullable until pledged)
- Request 1:N RequestStatusHistory (full audit trail); Request 1:1 Rating (only after COMPLETED)
- Documents table is polymorphic (owner_type + owner_id, no FK constraint) so one table serves Homes, Providers, and Request images alike

6. Authentication & Security

6.1 Login flow

- Standard accounts: username/password -> JWT issued immediately
- Administrator accounts: password verified first, then a 6-digit code emailed (5-minute expiry) as a second factor before a JWT is issued — true multi-factor, not just an extra password

6.2 Email verification — a hard gate

Registering doesn't block login, but verification IS required before: registering/resubmitting a Home or Provider profile, creating a request, any request status change (pledge, accept, progress, deliver, complete, cancel), and submitting a rating. Enforced server-side via `CurrentUserResolver.getCurrentVerifiedUser()`, and mirrored on the frontend by disabling the relevant buttons with an inline explanation.

6.3 Brute-force protection

Five failed login attempts on the same identifier within 15 minutes triggers a temporary logout on further attempts and notifies every Administrator — closing what was originally a gap where only password-reset/verification-resend were rate-limited.

6.4 Document encryption at rest

Every uploaded file (registration certificates, police clearance reports, request images) is encrypted with AES-256-GCM before being written to disk, and decrypted on retrieval. The encryption key lives only in an environment variable, never in code or the database.

6.5 Other measures

- Passwords hashed with BCrypt, never logged or returned in any response
- Role-based access control enforced server-side at both the URL-pattern level (`SecurityConfig`) and the method level (`@PreAuthorize`) — never trusted from the frontend alone
- Verification/reset/MFA tokens stored as SHA-256 hashes, single-use, short expiries
- Enumeration-safe responses on login and forgot-password (identical response whether or not the account exists)
- Suspended accounts lose access immediately — even an already-issued JWT is rejected on the next request, not just at the next login

- CORS locked to known frontend origins, never a wildcard

7. Core Modules

Module	What it does
Auth & RBAC	Registration, login, JWT, five roles (Donor, Delivery Volunteer, Service Provider, Children's Home, Administrator)
Verification	Document-backed approval workflow for Homes and Providers, including mandatory police clearance for onsite Providers
Resubmission	A rejected Home/Provider profile can be corrected and resubmitted up to 3 times before requiring admin intervention
Document management	Upload/list/download with encryption at rest; owners can self-service-remove their own request images pre-pledge
Request lifecycle	6-state state machine (Created -> Pledged -> Accepted -> In Progress -> Delivered -> Completed, or Cancelled) with full history
Reputation	Ratings after completion; a low average score restricts future pledging
Admin oversight	User suspension/reinstatement, audit log, content flagging/moderation
Reports	Platform-wide statistics with CSV and PDF export
Directory	Children's Homes can browse Donor/Provider reputations (paginated, contact-details hidden)
Auto-match	"Recommended For You" — ranks open requests by the caller's past pledge categories, urgency as tiebreaker
Fraud detection	A bounded heuristic flags accounts with 3+ cancelled requests for admin review
Scheduled maintenance	Daily jobs: soft-deactivate accounts inactive 365+ days; remind Homes about stale unpledged requests and fulfillers about stalled pledges
Notifications	In-app + email on every significant event (see Section 9)

8. Email System

EmailService is a Java interface with four implementations, exactly one active at a time via a single environment variable (app.email-provider):

Provider	Mechanism	When to use
Resend	HTTPS API (default)	Simple, but the free sandbox sender only delivers to the account owner's own address until a domain is verified
SendGrid	HTTPS API	Single Sender Verification — verify one email address, no domain needed, sends to any recipient
Brevo	HTTPS API	Same single-sender model as SendGrid; alternative if SendGrid's signup process is the blocker

SMTP	Direct SMTP	Self-hosted/VPS only — most PaaS hosts (Render included) block outbound SMTP ports at the network level
------	-------------	---

All four implementations send asynchronously (@Async) — a slow or failing mail provider can never block or hang the HTTP request that triggered the email; failures are logged server-side, never surfaced as a stuck UI.

9. Notification System

A single entry point, `NotificationService.notify(...)`, is called by every other service that has something worth telling a user about. Each call creates both an in-app record (bell icon, dropdown, unread count, polled every 30 seconds) and an email, for: verification decisions, every request status change, ratings received, content flagged, account suspended/reinstated, and suspicious login activity.

10. API Overview

Full, always-current API documentation is served live by the backend at `/swagger-ui.html` — generated directly from the code, so it can't drift out of sync with a hand-written doc.

Base path	Covers
<code>/api/v1/auth</code>	Register, login, MFA, email verification, password reset
<code>/api/v1/childrens-homes</code>	Registration, profile, resubmission
<code>/api/v1/service-providers</code>	Registration, profile, resubmission
<code>/api/v1/documents</code>	Upload, list, download, self-service removal
<code>/api/v1/requests</code>	Full lifecycle, browsing, history, recommendations
<code>/api/v1/directory</code>	Browse Donors/Service Providers (Homes only)
<code>/api/v1/notifications</code>	List, unread count, mark read
<code>/api/v1/admin/*</code>	Verification queue, user management, moderation, reports, audit log
<code>/api/v1/public/*</code>	The only genuinely unauthenticated endpoints — featured requests for the homepage

11. Frontend Design System

- Brand palette: forest green primary, warm gold secondary accent, cream backgrounds, near-black-green for dark surfaces
- Space Grotesk for headings (loaded once via Google Fonts), system sans for body text
- A signature ambient "connection network" motif (soft linked nodes, gently pulsing) used consistently across the homepage hero, auth pages, and dashboard banner
- A themed modal system (`ModalContext/useModal`) replaces every native `window.confirm/prompt/alert` with a consistent in-app dialog

- Donut charts with a center total label, used throughout the dashboards for status breakdowns
- Fully responsive: sidebar collapses to a mobile drawer, list/card views wrap instead of overflowing on narrow screens, decorative graphics hide below relevant breakpoints

12. Testing

Backend: JUnit 5 + Mockito, no database required — focused on the highest-risk logic: the full request lifecycle state machine and role-permission rules, the reputation restriction threshold, and token hashing/expiry/replay-protection.

Frontend: Vitest + React Testing Library, covering the pagination component, status badge rendering, and AuthContext's role-checking logic.

Note: Coverage is deliberately targeted at logic most likely to silently break something important (permission checks, state transitions, security-sensitive token handling) rather than chasing exhaustive coverage of presentational code.

13. Deployment & Infrastructure

Production: Vercel (frontend, auto-deploy on push) + Render (backend, Docker-based) + Neon (Postgres). Self-hosting: a single docker-compose.yml brings up Postgres + backend + frontend + Caddy (automatic HTTPS) with one command — same codebase, no changes required.

14. Environment Configuration

Variable group	Purpose
DB_*	Postgres connection
JWT_SECRET, JWT_EXPIRATION_MS	Token signing
ADMIN_BOOTSTRAP_SECRET	Gates the one-time administrator provisioning endpoint
STORAGE_ENCRYPTION_KEY	AES-256 key for document encryption at rest
EMAIL_PROVIDER + RESEND_*/SENDGRID_*/BREV O_*/MAIL_*	Selects and configures the active email backend
FRONTEND_URL	Used to build links inside emails

15. Known Limitations & Next Steps

- File storage is local disk — fine for one instance, needs a cloud backend (S3-compatible) before running more than one backend instance
- The brute-force rate limiter is in-memory — correct for one instance, would need Redis for a multi-instance deployment
- No CI/CD pipeline — every deploy is a manual git push

- No automated migration tool (Flyway/Liquibase) — the numbered SQL files are applied by hand
- Notification delivery is polled (30s) rather than pushed via WebSockets — a reasonable tradeoff at current scale
- Test coverage is targeted, not exhaustive — several controllers and services have no automated tests yet

None of these are defects in what's built — they're the natural next investments if this moves from a working build to something serving real, sustained traffic.